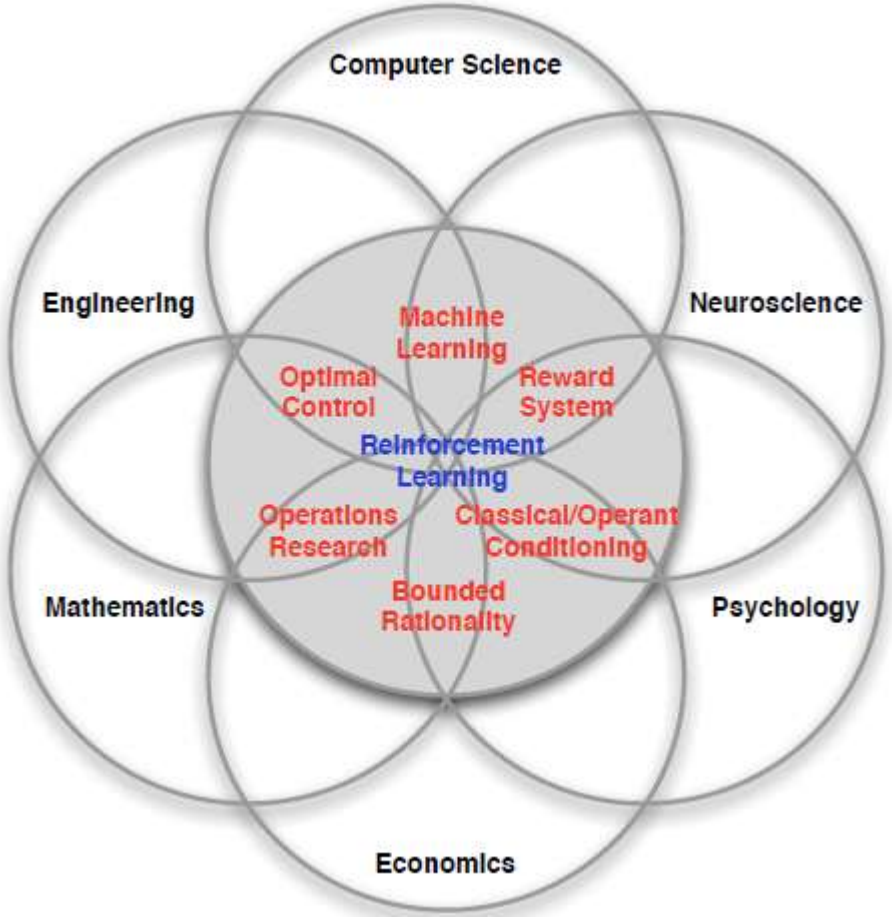
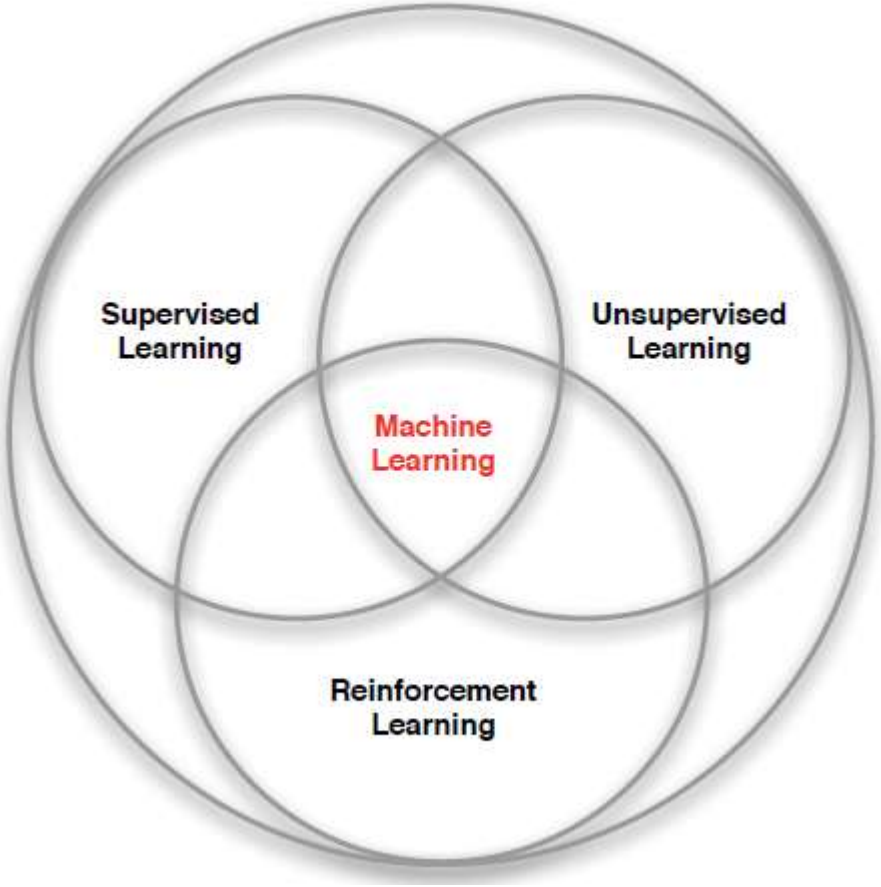
About RL

Reinforcement Learning?





- Many Faces
- One of ML types

About RL

Characteristics of Reinforcement Learning

- No supervisor, only a **reward** signal
- **Feedback** is delayed, not instantaneous
- **Time** really matters (sequential, non i.i.d data)
  - i.i.d -> independent and identically distributed
  - A sequence of dice rolls
- Agent's **actions** affect the subsequent data it receives

About RL

Examples of Reinforcement Learning

- Fly stunt manoeuvres in a helicopter
- Defeat the world champion at Backgammon
- Manage an investment portfolio
- Control a power station
- Make a humanoid robot walk
- Play many different Atari games better than humans

The RL Problem

Rewards

- A **reward** R_t is a scaler feedback signal
- Indicates **how well** agent is doing at step t
- The agent's job is to maximize **cumulative reward**

Reward hypothesis

- RL is based on the reward hypothesis
- Def: All **goals** can be described by the **maximization of expected cumulative reward**

The RL Problem

Examples of Rewards

- Fly stunt manoeuvres in a helicopter
  - +ve reward for following desired trajectory
  - -ve reward for crashing
- Manage an investment portfolio
  - +ve reward for each $ in bank
- Make a humanoid robot walk
  - +ve reward for forward motion
  - -ve reward for falling over
- Play many different Atrai games better than humans
  - +/-ve reward for increasing/decreasing score

The RL Problem

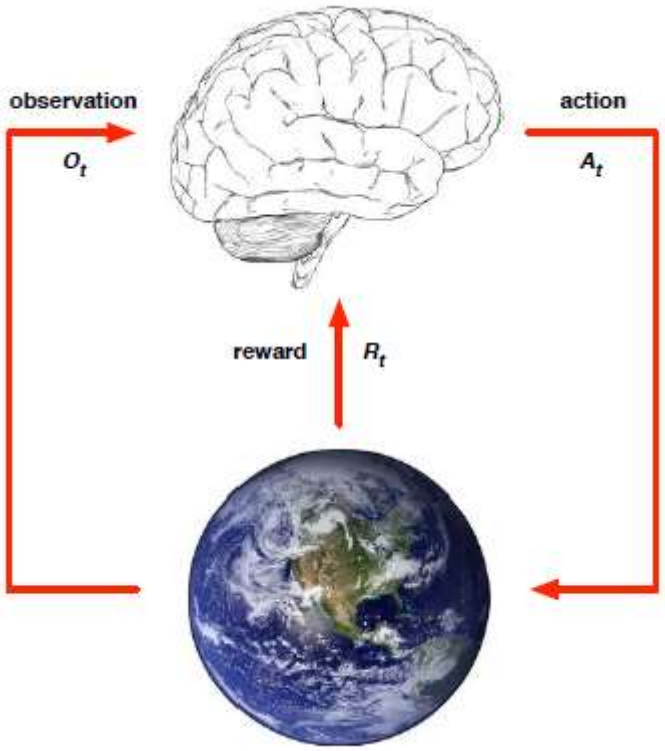
Sequential Decision Making

- Goal: **select actions to maximize total future reward**
- Actions may have **long term consequences**
- Reward may be **delayed**
- It may be better to **sacrifice immediate reward** to gain **more long-term reward**
- **Examples**
  - A financial investment (may take months to mature)
  - Refueling a helicopter (might prevent a crash in several hours)

The RL Problem

Agent and Environment

- At each step t the agent
  - Executes action A_t
  - Receives observation O_t
  - Receives scalar reward R_t
- The environment
  - Receives action A_t
  - Emits observation O_{t+1}
  - Emits scalar reward R_{t+1}
- t increments at env. step



The RL Problem

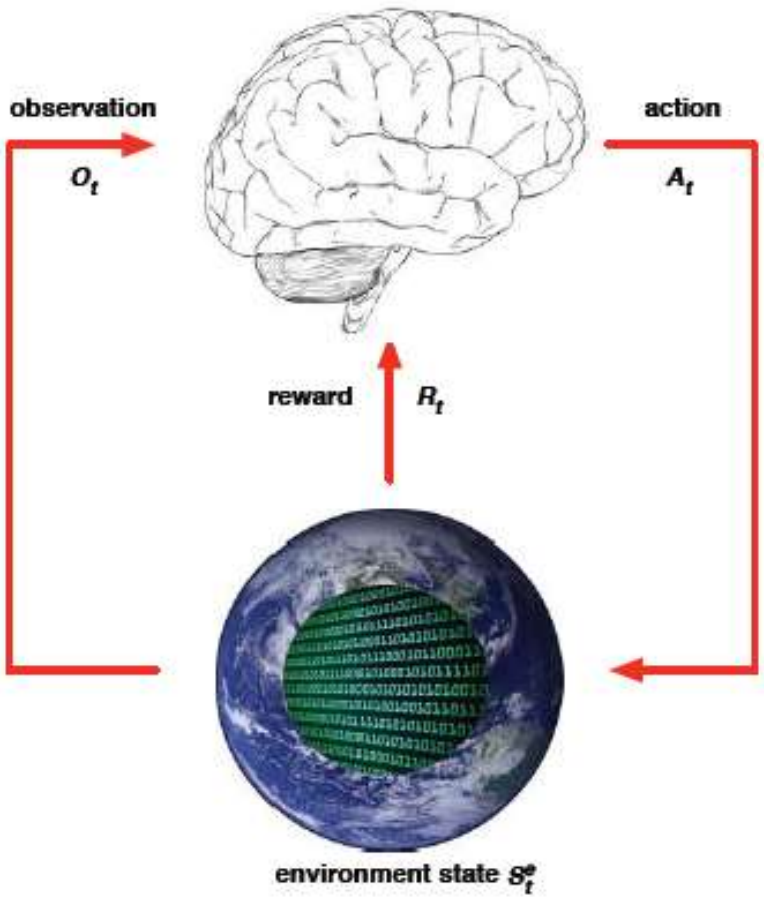
History and State

- The history is the sequence of observations, actions, rewards
  - $H_t = O_1, R_1, A_1, \dots, A_{t-1}, O_t, R_t$
  - All observable variables up to time t
- What happens next depends on the history
  - The agent selects actions
  - The environment selects observations/rewards
- **State** is the information used to determine **what happens next**
- Formally, state is a function of the history
  - $S_t = f(H_t)$

The RL Problem

Environment State

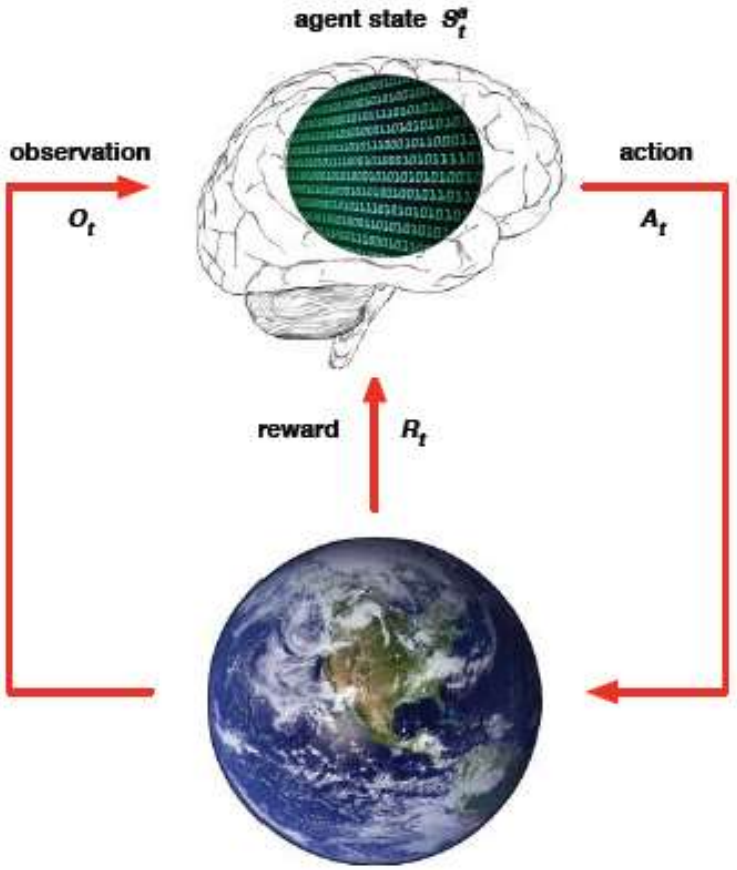
- The **environment state** S_t^e is the environment's private representation
  - Whatever data the environment uses to pick the **next observation/rewa**rd
- The environment state is **not usually visible** to the **agent**
- Even if S_t^e is visible, it may contain irrelevant information



The RL Problem

Agent State

- The agent state S_t^a is the agent's internal representation
  - Whatever information the agent uses to pick the **next action**
  - It is the information used by reinforcement learning algorithms
- It can be any function of history
  - $S_t^a = f(H_t)$



The RL Problem

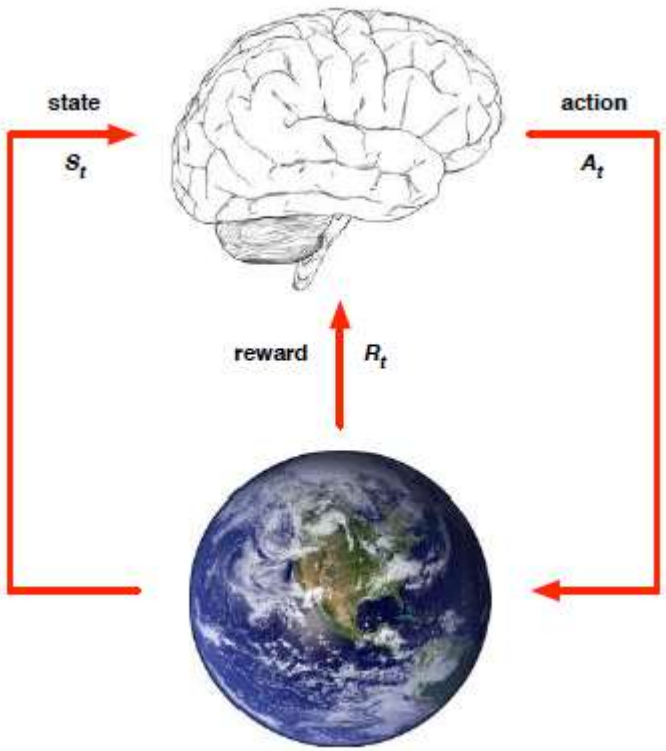
Information State (a.k.a. Markov state)

- A state S_t is **Markov** if and only if
  - $\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_1, \dots, S_t]$
- The future is independent of the past given the present
- Once the state is known, the history may be thrown away
  - The state is a sufficient statistic of the future
- The environment state S_t^e is Markov
- The history H_t is Markov

The RL Problem

Fully Observable Environments

- Fully observability: agent **directly** observes environment state
  - $O_t = S_t^a = S_t^e$
- Agent state = environment state = information state
- Formally, this is a **Markov decision process (MDP)**



The RL Problem

Partially Observable Environments

- Partial observability: agent **indirectly** observes environment state
  - A robot with camera vision isn't told its absolute location
  - A trading agent only observes current prices
  - A poker playing agent only observes public cards
- Agent state != environment state
- Formally, this is a **partially observable Markov decision process (POMDP)**
- Agent must construct its own state representation S_t^a
  - Complete history: $S_t^a = H_t$
  - Beliefs of environment state: $S_t^a = (\mathbb{P}[S_t^e = S^1], \dots, \mathbb{P}[S_t^e = S^n])$
  - Recurrent neural network: $S_t^a = \sigma(S_{t-1}^a W_s + O_t W_o)$

Inside An RL Agent

Major Components of an RL Agent

- An RL agent may include one or more of these components
  - Policy: agent's behavior function
  - Value function: how good is each state and/or action
  - Model: agent's representation of the environment

Inside An RL Agent

Policy

- It is a map from state to action
  - Deterministic policy: $a = \pi(s)$
  - Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$

Value Function

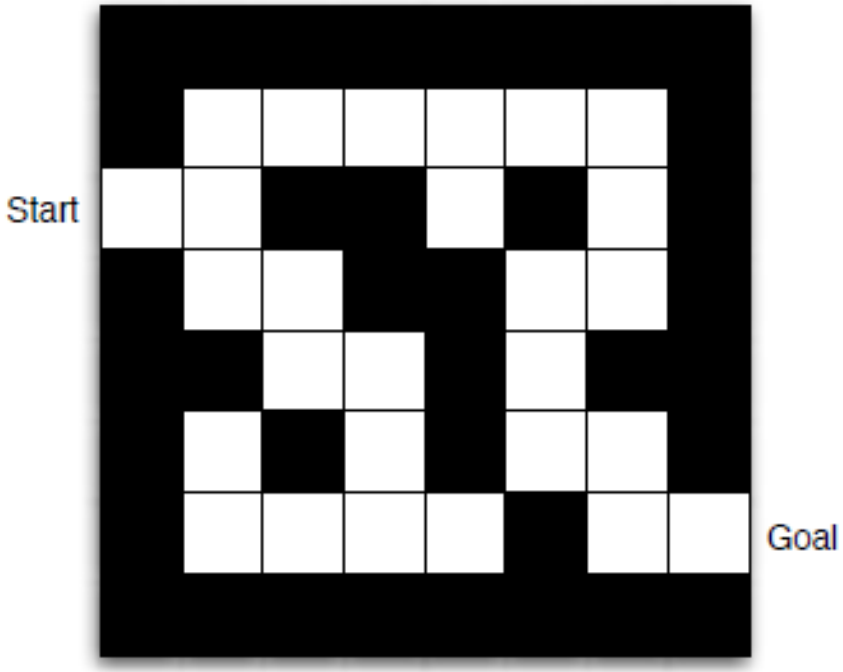
- Value function is a prediction of future reward
- Used to evaluate the goodness/badness of states
  - $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots |S_t = s]$

Model

- A model predicts what the environment will do next
- P predicts the next state, R predicts the next (immediate) reward

Inside An RL Agent

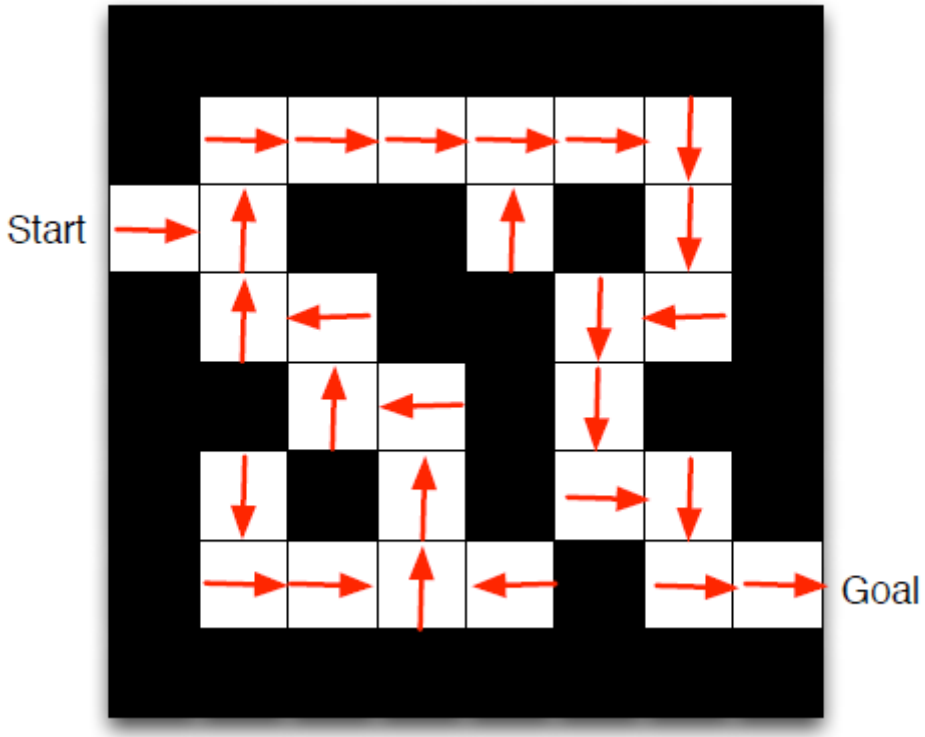
Maze Example



- Rewards: -1 per time-step
- Actions: N, E, S, W
- States: Agent's location

Inside An RL Agent

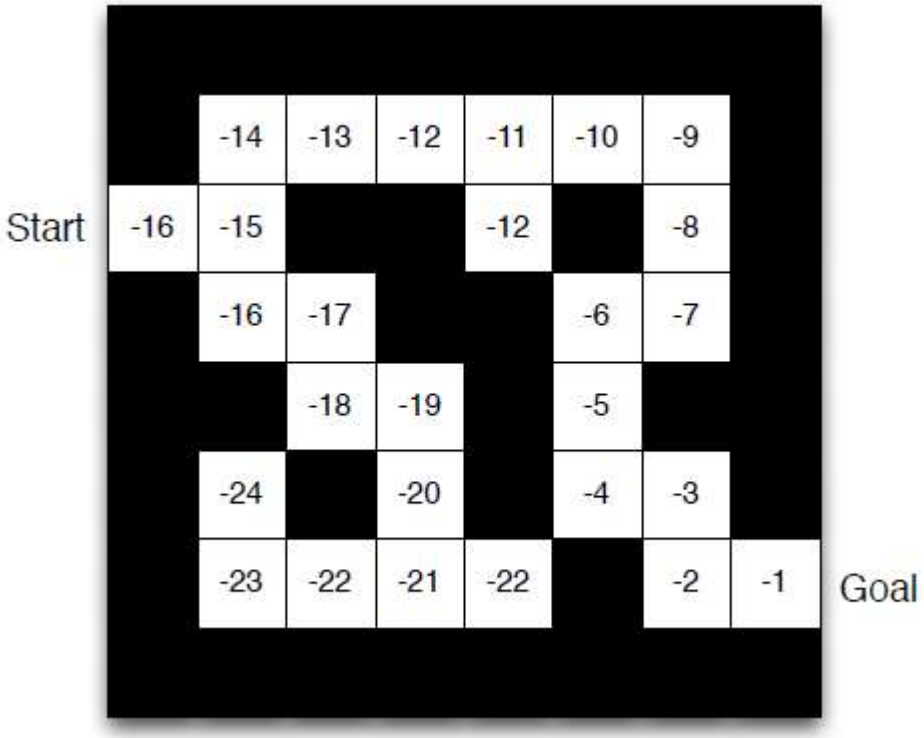
Maze Example: Policy



- Map state (location) to action

Inside An RL Agent

Maze Example: Value Function

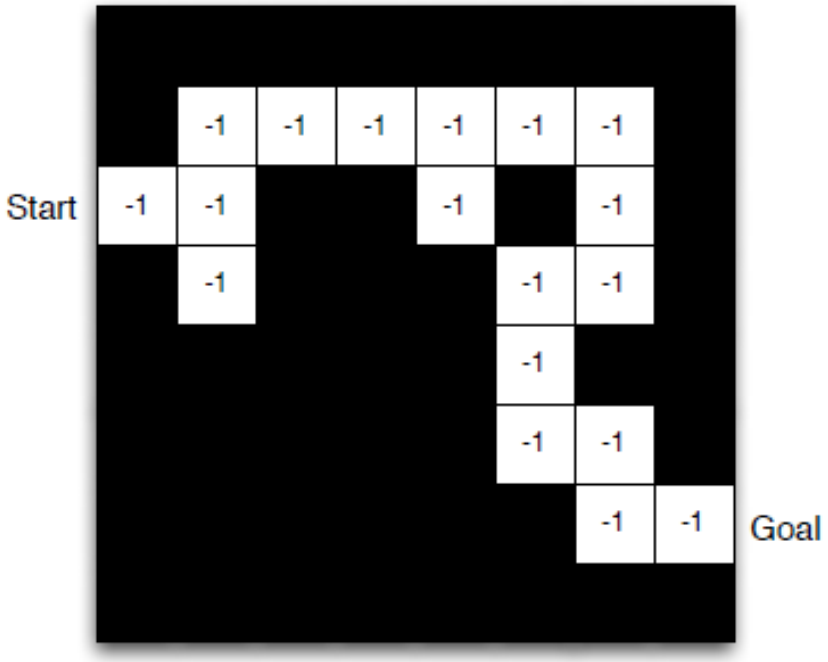


- Numbers represent value of each state s

- Value function is a prediction of future reward
- Used to evaluate the goodness/badness of states
  - $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s]$

Inside An RL Agent

Maze Example: Model



- Agent may have an internal model of the environment
- Dynamics: how actions change the state
- Rewards: how much reward from each state
- The model may be imperfect

- A model predicts what the environment will do next
- P predicts the next state, R predicts the next (immediate) reward

Inside An RL Agent

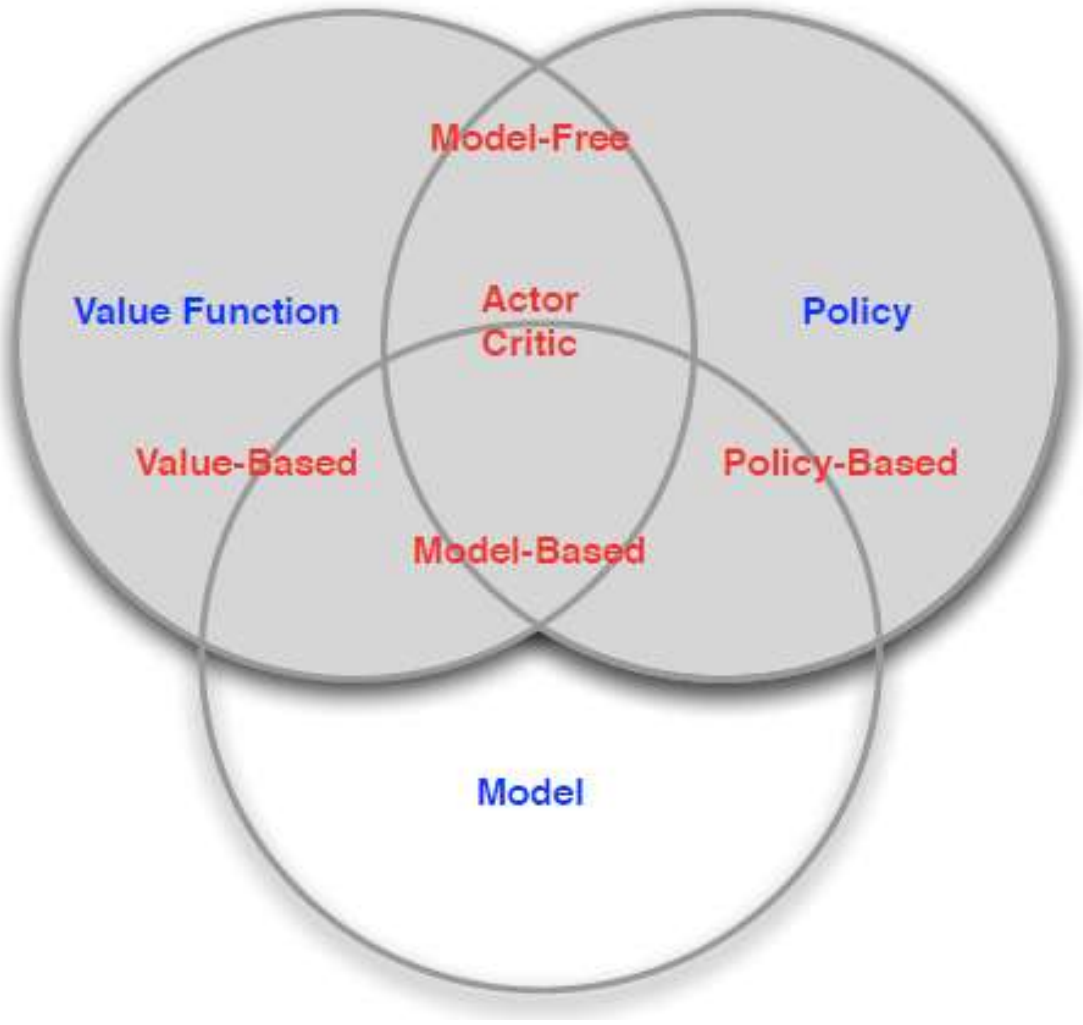
Categorizing RL agents (1)

- Value Based
  - No policy (Implicit)
  - Value Function
- Policy Based
  - Policy
  - No Value Function
- Actor Critic
  - Policy
  - Value Function

Inside An RL Agent

Categorizing RL agents (2)

- Model Free
  - Policy and/or Value Function
  - No Model
- Model Based
  - Policy and/or Value Function
  - Model



Problems within RL

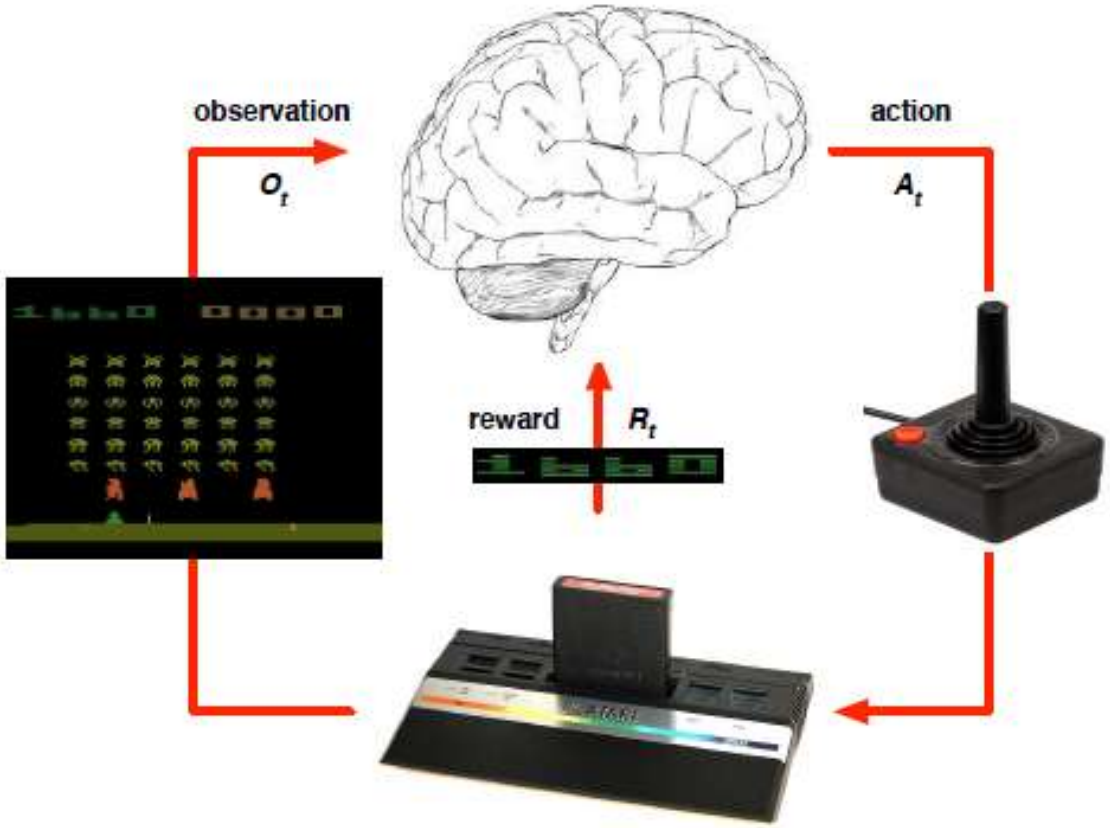
Learning and Planning

- Reinforcement Learning
  - The environment is initially unknown
  - The agent interacts with the environment
  - The agent improves its policy
- Planning
  - A model of the environment is known
  - The agent performs computations with its model
  - The agent improves its policy
  - A.k.a. deliberation, reasoning, introspection, pondering, thought, search

Problems within RL

Atari Example: Reinforcement Learning

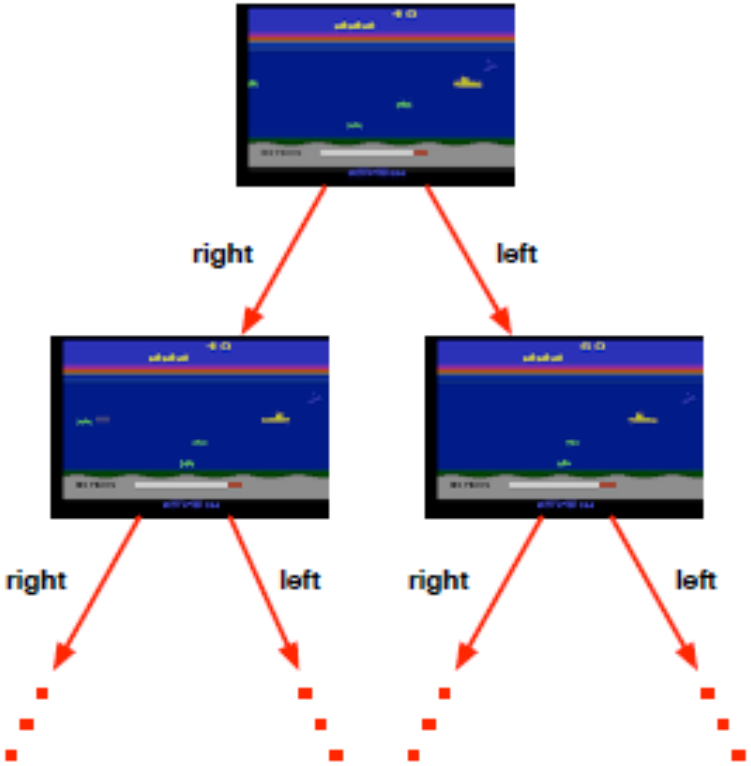
- Rules of the game are unknown
- Learn directly from interactive game-play
- Pick actions on joystick, see pixels and scores



Problems within RL

Atari Example: Planning

- Rules of the game are known
- Can query emulator
- If I take action a from state s
  - What would the next state be?
  - What would the score be?
- Plan ahead to find optimal policy
  - E.g. tree search



Problems within RL

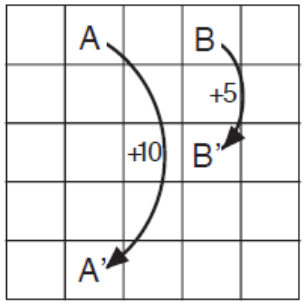
Exploration and Exploitation

- Reinforcement learning is like trial-and-error learning
- The agent should discover a good policy
- From its experience of the environment
- **Exploration** finds more information about the environment
- **Exploitation** exploits known information to maximize reward
- It is usually important to explore as well as exploit
  - Online banner advertisements
    - Exploitation -> Show the most successful advert
    - Exploration -> Show a different advert

Problems within RL

Prediction and Control

- Prediction -> evaluate the future (Given a policy)



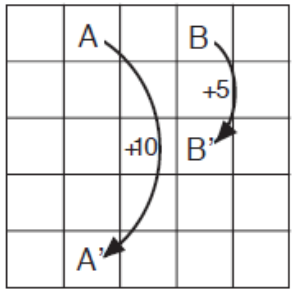
(a)

Actions

3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

(b)

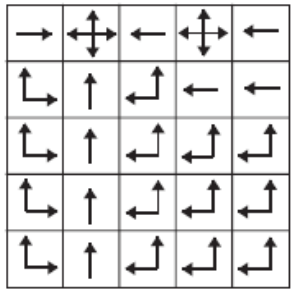
- Control -> optimize the future (Find the best policy)



a) gridworld

22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

b) v_*



c) π_*